DO NOW — RECALL

# Modularity, functions & procedures

Name: _____ Class: _____ Date: _____

**1. Which loop is most suitable when the number of repetitions is known in advance?**

**2. In C#, what does && mean, and when is the whole expression true?**

**3. What is the largest unsigned 8-bit denary value?**

**4. A program reads a mark, repeats through five scores and uses if to count passes. Name the three programming constructs present.**

DO NOW — CHECK / OBJECTIVES

# Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

**1. Model answer**

A for loop . It is commonly used for count-controlled iteration.

**2. Model answer**

&& means AND . The whole expression is true only when both Boolean expressions are true.

**3. Model answer**

255 Eight bits give 256 patterns, representing values 0–255.

**4. Model answer**

Sequence · iteration · branching/selection Instructions happen in order, a loop repeats code, and if chooses whether the pass counter changes.

**By the end, you can…**

• explain why programs are split into smaller modules;

• write and call a C# procedure using void ;

• write and call a C# function that returns a value.

**Vocabulary — know these terms**

Recall the term — Modularity

Recall the term — Subroutine

Recall the term — Method

Recall the term — Function

Recall the term — Procedure

Recall the term — Return value

Recall the term — Parameter

Recall the term — Argument

NEW LEARNING

# Read, annotate and explain

### Methods sit outside Main

```
public class Program
{
public static void Main()
{
Greeting("Aisha");
int answer = Triple(7);
}

public static void Greeting(string name)
{
Console.WriteLine("Hello " + name);
}

public static int Triple(int number)
{
return number * 3;
}
}
```

OCR language: a function returns a value. A procedure does not. In C#, both are methods.

### Modularity breaks a problem into parts

A large program is easier to understand when separate jobs are placed in separate subroutines.

### A procedure performs a job

A C# procedure-style method uses void because it does not return a value to the caller.

### A function returns a value

The return type replaces void . A return statement sends the result back to the calling code.

### Parameters receive data

A parameter is the named input in the method definition. An argument is the actual value supplied when the method is called.

**Explain one step in your own words, using the correct technical terms.**

I DO — WORKED EXAMPLE

# Follow the worked method

### Build an IsEven function

### Choose the return type

```
public static bool IsEven(int number)
{
}
```

The result is true/false, so the return type is bool .

### Use the remainder test

```
public static bool IsEven(int number)
{
    return number % 2 == 0;
}
```

The Boolean expression itself already produces true or false .

### Use it from Main

```
public static void Main()
{
    bool result = IsEven(12);
    Console.WriteLine(result);
}
```

The argument 12 is received by the parameter number .

### Use contrasting values

```
IsEven(12) → True
IsEven(7)  → False
IsEven(0)  → True
```





WE DO — CHECK AND IMPROVE

# Source-grounded model answers

CHECK only after completing the guided practice and Exit Ticket. Use GREEN PEN to improve your method and explain any error.

### Guided 1 — CHECK

Function or procedure?

### Guided 2 — CHECK

Complete the method
CODE:
public static ? Square(int number)
{
?
}
CODE:
public static int Square(int number)
{
return number * number;
}

### Guided 3 — CHECK

Design the module

### Guided 4 — CHECK

Complete both statements

EXIT TICKET / REFLECTION

# Check your understanding

Complete the actual lesson Exit Ticket before checking.

**A function returns a value to its caller.**

**Circle: Confident / Getting there / Need help**

Explain what you can now do independently.

**GREEN PEN — Correct and explain**

Choose one error. Write the correct method and explain precisely why your first response was wrong.





WE DO — GUIDED PRACTICE

# Try → Discuss → Improve

TRY: Attempt the guided questions in BLACK PEN. DISCUSS: Compare approaches with a partner, explain your reasoning and agree one improvement. IMPROVE: After the Exit Ticket, use the source CHECK model in GREEN PEN to correct and explain errors.

**Guided 1 — TRY: Greeting procedure**

Create a method called Greeting that receives a name and displays Hello name .

Method / working:

**Guided 2 — TRY: Square function**

Create a method that receives an integer and returns its square.

```
public static ? Square(int number)
{
    ?
}
```

Method / working:

**Guided 3 — TRY: Refactor repeated code**

A program prints the same separator in several places: Move that repeated job into one method.

Method / working:

**Guided 4 — TRY: Function or procedure?**

OCR asks students to distinguish the two. State the difference precisely.

Method / working:

YOU DO — INDEPENDENT PRACTICE

# Questions · Part 1

Answer independently. Show code, calculations or a justified explanation.

### Question 1

DisplayTitle procedure
Create a procedure-style method called DisplayTitle() .
It should output OCR H446 Computer Science .
Call it from Main() twice.

### Question 2

DoubleIt function
Create DoubleIt(int number) that returns twice the value supplied.
Call the function with 12 , store the returned value, then output it.





YOU DO — INDEPENDENT PRACTICE

# Questions · Part 2

Answer independently. Show code, calculations or a justified explanation.

### Question 3

IsPass function
Create IsPass(int mark) .
Return true when the mark is at least 50; otherwise return false .
Call it from Main() with at least two different marks.

### Question 4

Refactor a program into modules
Write a program that asks for two integers and then:
1. calls Add(int first, int second) and outputs the returned total;
2. calls ShowLine() before and after the result.